



UNIVERSIDAD DIEGO PORTALES

Laboratorio 3: Ordenamiento y Búsqueda

Problema: Base de datos musical

ESTRUCTURAS DE DATOS Y ALGORITMOS
2026-1

Profesores:
Matías Aguilera
Marcos Fantoal
Cristián Llull
Karol Suchan

Trabajo en equipos de 3 estudiantes

1. Introducción

En este laboratorio, usted y su equipo deberán implementar algoritmos utilizados en infinidad de áreas en el mundo de la informática: algoritmos de búsqueda y ordenamiento. El objetivo es experimentar cómo funcionan las bases de datos relacionales y, en este caso particular, una de música.

1.1. Motivación

Las plataformas de música en *streaming*, como *Spotify* o *Apple Music*, gestionan catálogos con decenas de millones de canciones. Cada vez que un usuario ordena su biblioteca por artista, busca una canción por nombre o filtra por género, hay algoritmos de ordenamiento y de búsqueda que operan sobre grandes volúmenes de datos.

En el mundo del desarrollo de software, estas operaciones suelen expresarse mediante consultas a bases de datos. Por ejemplo:

```
SELECT * FROM songs ORDER BY plays DESC
SELECT * FROM songs WHERE artist = 'The Weeknd'
```

Aunque en este laboratorio no trabajaremos directamente con SQL (*Structured Query Language*) ni con motores de bases de datos, estas consultas nos sirven de motivación: cada una de ellas requiere, internamente, algún algoritmo de ordenamiento o de búsqueda. El objetivo de este laboratorio es estudiar empíricamente y teóricamente cómo se comportan distintos algoritmos al resolver tareas equivalentes a dichas consultas, sobre conjuntos de datos de tamaño creciente.

1.2. Objetivos

- Implementar una clase `Song` y otra `SongDataBase` que modelen una base de datos de música simplificada.
- Adaptar los algoritmos de ordenamiento y búsqueda del libro guía de Sedgewick y Wayne para que operen sobre objetos `Song`,
- Comparar los resultados empíricos con el análisis teórico Big-O.

2. Especificaciones de las clases

Deberá implementar las clases `Song`, `SongDataBase` y `DataGenerator` desde cero.

2.1. Clase `Song`

La clase `Song` representa una canción con los siguientes atributos:

Atributo	Tipo	Descripción
<code>'id'</code>	<code>'int'</code>	Identificador único de la canción
<code>'title'</code>	<code>'String'</code>	Título de la canción
<code>'artist'</code>	<code>'String'</code>	Nombre del artista
<code>'genre'</code>	<code>'String'</code>	Género musical
<code>'year'</code>	<code>'int'</code>	Año de lanzamiento
<code>'plays'</code>	<code>'long'</code>	Número de reproducciones

Con los siguientes métodos:

- `Song(int id, String title, String artist, String genre, int year, long plays):`
- Getters para cada atributo.

- Para comparar objetos de tipo `Song` durante el ordenamiento, deberá implementar los comparadores necesarios. Puede usar objetos de tipo `Comparator` o bien hacer que la clase `Song` implemente `Comparable`.

2.2. Clase `SongDataBase`

La clase `SongDataBase` gestiona la colección de canciones y proporciona métodos para ordenarlas y buscarlas.

Como único atributo, esta clase tiene `ArrayList<Song> songs`, que representa una lista de canciones de la base de datos.

Para los métodos, debe implementar los declarados en las siguientes subsecciones: ordenamiento y búsqueda.

2.2.1. Ordenamiento

`ordenarPorAlgoritmo(String algoritmo, String atributo)`: ordena la base de datos según el algoritmo y el atributo indicados.

- `algoritmo` puede tomar los siguientes valores: `'insertionSort'`, `'selectionSort'`, `'mergeSort'` o `'quickSort'`. Si no coincide con ninguno, usar `Collections.sort()`.
- `atributo` puede tomar los siguientes valores: `'plays'`, `'artist'`, `'genre'` o `'year'`. Si no se reconoce, ordenar por `'plays'` por defecto.

2.2.2. Búsqueda

- `sequentialSearch(String artista)`: recorre la lista de canciones de inicio a fin y retorna un `'ArrayList<Song>'` con todas las canciones cuyo artista coincida exactamente con el parámetro. No requiere que la lista esté ordenada.
- `binarySearch(String artista)`: busca en la lista asumiendo que está **ordenada por artista**. Debe encontrar el primer índice en el que aparece el artista buscado y luego recorrer hacia adelante hasta que el artista cambie. Retorna un `'ArrayList<Song>'` con todas las coincidencias.

El método `binarySearch` debe asumir que la lista `songs` está ordenada por `artist`; es responsabilidad del experimento garantizar que esté en ese orden antes de llamarlo. Debe inspirarse en el siguiente método `rank`, que en un arreglo ordenado de forma creciente encuentra la cantidad de elementos estrictamente menores que `key`. Para el método `sequentialSearch` debe hacer una implementación propia.

```

1 public static <T extends Comparable<? super T>> int rank(T[] data, T key) {
2     int lo = 0;
3     int hi = data.length;
4     while (lo < hi) {
5         int mid = lo + (hi - lo) / 2;
6         if (data[mid].compareTo(key) < 0) {
7             lo = mid + 1;
8         } else {
9             hi = mid;
10        }
11    }
12    return lo;
13 }
```

2.3. Algoritmos de ordenamiento

Modifique las clases `Insertion`¹, `Selection`², `Merge`³ y `Quick`⁴ del libro guía, las cuales operan sobre arreglos de tipos genéricos, para que utilicen `ArrayList<Song>`, recibiendo un `Comparator<Song>` que determine el criterio de ordenamiento. El método principal de cada clase modificada debe ser:

```
1 public static void sort(ArrayList<Song> songs, Comparator<Song> comparator)
```

La lógica central del algoritmo debe mantenerse fiel al original. No debe reimplementar el algoritmo, sino realizar los ajustes mínimos necesarios para que opere con la nueva estructura de datos.

2.4. Algoritmos de búsqueda

Utilice como inspiración el método `rank` entregado en la sección anterior para el algoritmo de búsqueda binaria. Para la búsqueda lineal, desarrolle su propia implementación.

3. Generación de datos sintéticos

Implemente una clase `DataGenerator` con un método estático:

```
1 public static ArrayList<Song> generateDataBase(int n, long seed)
```

Este método debe:

1. Usar `StdRandom.setSeed(seed)` para fijar la semilla.
2. Definir internamente un arreglo fijo de artistas de tamaño $n/50$. Por ejemplo, para $n = 1024$ habrá 20 artistas ('Artista_0', ... 'Artista_19'). Esto asegura que siempre haya varias canciones de cada artista.
3. Generar n objetos tipo `Song` con los siguientes criterios:
 - **id**: entero secuencial de 1 a n .
 - **title**: string de la forma 'Song_+id'.
 - **artist**: seleccionado aleatoriamente desde el arreglo de artistas del paso anterior.
 - **genre**: seleccionado aleatoriamente desde un arreglo fijo con los siguientes elementos: 'Pop', 'Rock', 'Jazz', 'Electronic', 'Classical', 'Hip-Hop'.
 - **year**: entero aleatorio en el rango [1970, 2026].
 - **plays**: entero aleatorio en el rango [0, 10,000,000].
4. Retornar un `ArrayList<Song>` con las canciones generadas.

Utilice `seed = n + i`, donde i es el número de la instancia (comenzando en 0). Esto garantiza que cada instancia sea distinta y que los experimentos sean reproducibles.

4. Etapas del Laboratorio

4.1. Etapa 1: Implementación

Implemente las clases `Song`, `SongDataBase` y `DataGenerator` según la especificación de las secciones anteriores.

Implemente también una clase en 'Experimento.java' que ejecute los dos experimentos descritos en las secciones 4.2 y 4.3, y que exporte los resultados a archivos CSV usando la clase `Out` de Princeton⁵.

¹<https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/Insertion.java.html>

²<https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/Selection.java.html>

³<https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/Merge.java.html>

⁴<https://algs4.cs.princeton.edu/code/edu/princeton/cs/algs4/Quick.java.html>

⁵<https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Out.html>

4.2. Etapa 2: Experimento 1 - Algoritmos de ordenamiento

El objetivo es comparar empíricamente el tiempo de ejecución de los cuatro algoritmos de ordenamiento (*Insertion Sort*, *Selection Sort*, *Merge Sort* y *Quick Sort*) sobre los mismos conjuntos de datos, utilizando la estrategia de Doubling Ratio.

4.2.1. Procedimiento

Para cada tamaño $n = 2^t$, con $t \in \{10, 11, 12, 13, 14, 15\}$, es decir: 1024, 2048, 4096, 8192, 16384, 32768, repita el siguiente proceso 100 veces (una por instancia, con semilla $n + i$):

1. Genere una base de datos de n canciones usando `generateDatabase(n, n+i)`.
2. Para cada uno de los cuatro algoritmos de ordenamiento:
 - a) Cree una copia independiente del `ArrayList<Song>` original.
 - b) Cree un nuevo objeto `SongDataBase` a partir de esa copia.
 - c) Mida el tiempo que tarda `sortByAlgorithm(algoritmo, 'plays')` usando `StopWatchCPU`
3. Guarde los cuatro tiempos de la instancia en un archivo CSV.

El archivo CSV de cada tamaño debe tener el siguiente formato: instancia, insertionSort, selectionSort, mergeSort, quickSort.

Reglas para medir bien

- No imprimir en la consola durante las mediciones.
- No incluir la copia del `ArrayList` ni la creación del objeto `SongDataBase` dentro del bloque medido:
- Asegúrese de que cada experimento siempre parta del mismo estado desordenado.

4.3. Etapa 3: Experimento 2 - Búsqueda lineal vs. búsqueda binaria

El objetivo es medir y comparar el costo real de usar la búsqueda lineal sobre datos desordenados frente a la búsqueda binaria sobre datos ordenados.

Este experimento es análogo a la consulta SQL:

```
SELECT * FROM songs WHERE artist = 'X'
```

4.3.1. Artistas a buscar

Para cada tamaño n , el arreglo de artistas tiene $n/50$ elementos. Los 5 artistas buscados serán siempre los siguientes, seleccionados por índice fijo dentro del arreglo:

Artista	Índice en el arreglo
Artista 1	0 (primero)
Artista 2	$n/200$ (25 %)
Artista 3	$n/100$ (50 %)
Artista 4	$3 * n/200$ (75 %)
Artista 5	$n/50$ (último)

Por ejemplo, para $n = 1024$; los artistas buscados serían 'Artista_0', 'Artista_5', 'Artista_10', 'Artista_15' y 'Artista_19'.

4.3.2. Procedimiento

Para cada tamaño $n = 2^t$, con $t \in \{10, 11, 12, 13, 14, 15\}$, repita el siguiente proceso 100 veces (una por instancia, con semilla $n + i$):

1. Genere una base de datos de n canciones usando `generateDatabase(n, n+i)`.
2. Para cada uno de los 5 artistas fijos:
 - a) Búsqueda lineal:
 - 1) Sobre el objeto `SongDatabase` sin ordenar, ejecute `linearSearch(artist)` en un bucle de 1.000 repeticiones.
 - 2) Mida el tiempo total del bucle con `StopWatchCPU` y guárdelo en la variable `t_lineal`.
 - b) Búsqueda binaria:
 - 1) Cree una copia del objeto `SongDatabase`.
 - 2) Ordene los datos usando uno de los algoritmos de ordenamiento y el atributo 'artista'.
 - 3) Inicie la medición de tiempo con `StopWatchCPU`.
 - 4) Ejecute `binarySearch(artist)` en el arreglo ordenado.
 - 5) Repita el procedimiento 1.000 veces.
 - 6) Guarde el valor del tiempo obtenido en la variable `t_binaria`.
3. Guarde los tiempos en un archivo CSV. Este archivo debe tener el siguiente formato: 'instancia', 'artista', 't_lineal', 't_binaria'.

4.4. Etapa 4: Análisis teórico y Comparación

4.4.1. Excel

Debe entregar un archivo `analisis.xlsx` con la siguiente estructura:

1. Seis pestañas de ordenamiento, denominadas 'Sort_1024', 'Sort_2048', ... 'Sort_32768'.
2. Seis pestañas de búsqueda, denominadas 'Sort_1024', 'Sort_2048', ... 'Sort_32768'.
3. Una pestaña final llamada **Resumen**.

En cada pestaña de tamaño, para cada método, incluya:

1. Los tiempos medidos (microdatos).
2. Estadísticas básicas:
 - a. Mínimo
 - b. Máximo
 - c. Promedio
 - d. Desviación estándar
 - e. Coeficiente de variación (desviación estándar / promedio)
3. Un histograma de los tiempos.

Qué debe incluir la pestaña Resumen:

1. Una tabla con los **promedios** por tamaño de cada método (para ambos experimentos).
2. Un gráfico **log-log** para el experimento 1 con cuatro curvas (una por algoritmo):
 - Eje x : tamaño n .

- Eje y : tiempo promedio.
3. Un gráfico **log-log** para el experimento 2 con dos curvas (una por algoritmo):
 - a) Eje x : tamaño n .
 - b) Eje y : tiempo promedio.
 4. Cálculo de ratios entre tamaños consecutivos y sus logaritmos en base 2.
 5. Interpretación breve: ¿a qué valor parece converger el logaritmo para cada algoritmo?

4.4.2. Análisis teórico

Para cada algoritmo de ordenamiento (*Insertion Sort*, *Selection Sort*, *Merge Sort*, *Quick Sort*) y para cada estrategia de búsqueda (lineal, binaria), debe:

1. Explicar brevemente cómo funciona (idea general).
2. Identificar las operaciones que dominan el tiempo de ejecución.
3. Justificar su complejidad en notación Big-O.

Debe quedar claro:

- Qué operación se repite más veces.
- Qué término domina cuando $n \rightarrow \infty$.
- Por qué se ignoran constantes y términos menores.

4.4.3. Comparación de teoría con experimentación

Para cada algoritmo, compare:

1. ¿El orden de crecimiento observado en el gráfico log-log coincide con la complejidad Big-O teórica?
2. ¿Los valores $\log_2(\text{ratio})$ se acercan al exponente esperado?
3. Para el experimento 2: ¿qué fracción del costo total de la búsqueda binaria corresponde al ordenamiento? ¿En qué escenarios convendría pagar ese costo?

5. Informe

Además del código y el Excel, deben entregar un **informe en PDF** que documente todo el proyecto.

5.1. Estructura obligatoria

El informe debe incluir, en este orden:

1. Introducción

- Descripción general del desafío.
- Objetivos del laboratorio.
- Qué entregables se construyeron (visión general).

2. Etapa 1: Implementación

- Qué clases tiene su programa y para qué sirve cada una.

- Decisiones de diseño: cómo se implementaron los comparadores, cómo funciona la búsqueda binaria con múltiples resultados.

3. Etapas 2 y 3: Análisis empírico

(I) Estructura del software

- Cómo se mide el tiempo y cómo se escriben los archivos CSV.

(II) Desafíos enfrentados

- Qué fue lo más difícil.
- Cómo lo resolvieron (criterios, pruebas, decisiones).

(III) Análisis empírico

- Qué muestran los promedios, histogramas, gráficos log-log.
- Qué concluyen con la estrategia de duplicación.

4. Etapa 4: Análisis teórico y Comparación

(I) Excel

- Qué contiene cada pestaña (tamaños).
- Qué contiene el Resumen y cómo leerlo.

(II) Análisis

- Big-O de cada método y su justificación.

(III) Comparación

- Tabla o resumen que compare la teoría con el experimento.
- Discusión argumentada.

5. Conclusiones

- Qué aprendieron.
- Qué algoritmo de ordenamiento escalaría mejor para una base de datos real y por qué.
- ¿En qué escenario real vale la pena el costo de ordenar los datos primero para poder usar la búsqueda binaria?
- Una breve reflexión sobre por qué conviene comparar teoría y práctica.

6. Entregables

Cada equipo debe entregar (vía Canvas) una carpeta comprimida **.zip** con:

1. Código fuente completo del programa Java para el análisis empírico.
2. Archivos CSV generados (uno por tamaño).
3. Archivo **analisis.xlsx**.
4. Informe en PDF.

Importante: si falta algún entregable, el laboratorio se considera incompleto.

Parte	Puntaje
Etapa 1: Programa Java y su correctitud	10
Etapa 1: Uso de clases de <i>Princeton</i> (StopwatchCPU, Out, modelo DoublingRatio)	5
Etapa 1: Modificación de clases de <i>Princeton</i> (correctitud y descripción de cambios en el informe)	10
Etapa 2 y 3: Excel por tamaño (microdatos + estadísticos + histograma)	5
Etapa 2 y 3: Resumen (tabla promedios + log-log + ratios + log2 + interpretación)	5
Etapa 4: Análisis teórico Big-O (justificación clara)	5
Etapa 4: Comparación teoría vs. experimento (discusión y conclusiones)	10
Informe (estructura, claridad, orden, redacción y evidencia)	10
Total	60

Cuadro 1: Rúbrica de evaluación del Laboratorio 2.

7. Rúbrica

8. Recomendaciones finales

1. Comience temprano: implementar cuatro algoritmos de ordenamiento, dos de búsqueda y el análisis en Excel requiere iteración.
2. Pruebe primero con tamaños pequeños (por ejemplo, $n = 32$, $n = 64$) para validar que su programa funciona antes de medir.
3. Asegúrese de que las copias del `ArrayList` sean independientes entre ejecuciones de distintos algoritmos: ordenar con InsertionSort y luego medir MergeSort sobre datos ya ordenados no constituye un experimento justo.
4. Evite imprimir durante la medición.
5. En el informe, privilegie explicaciones simples y concretas: qué hicieron, qué observaron y qué concluyen.